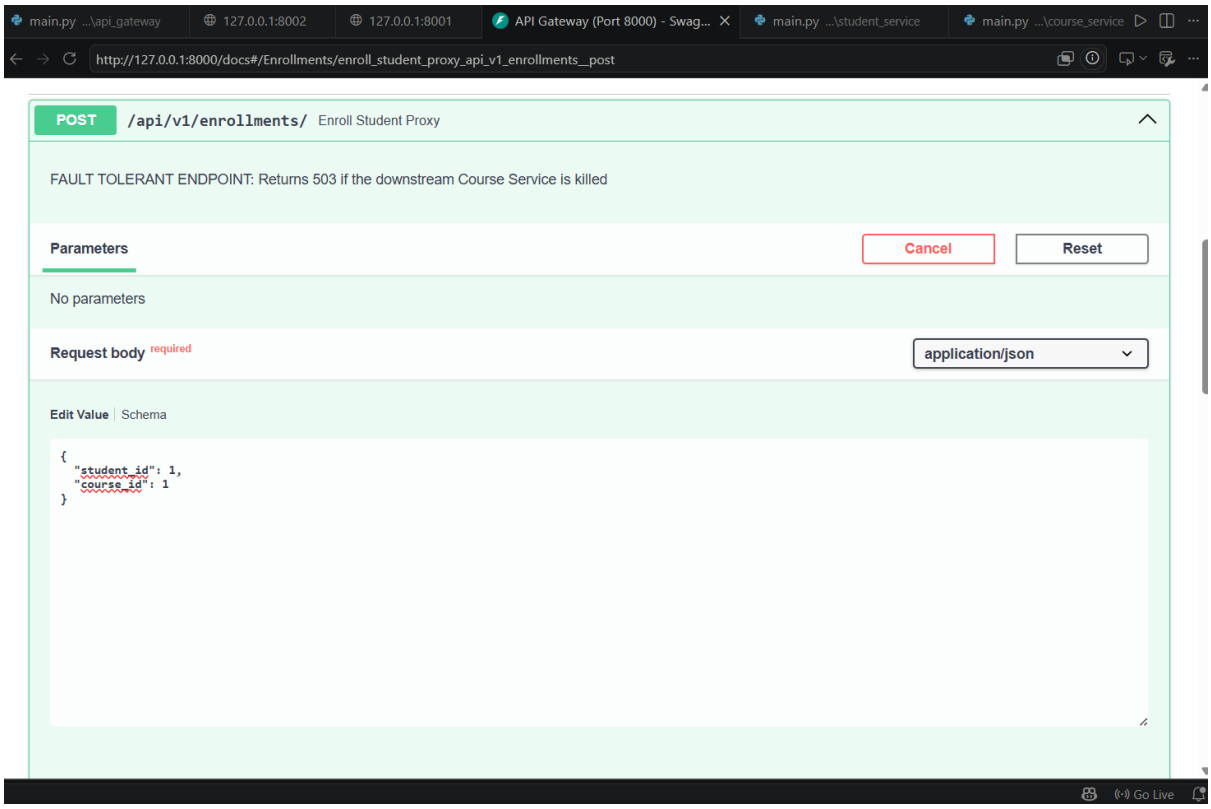
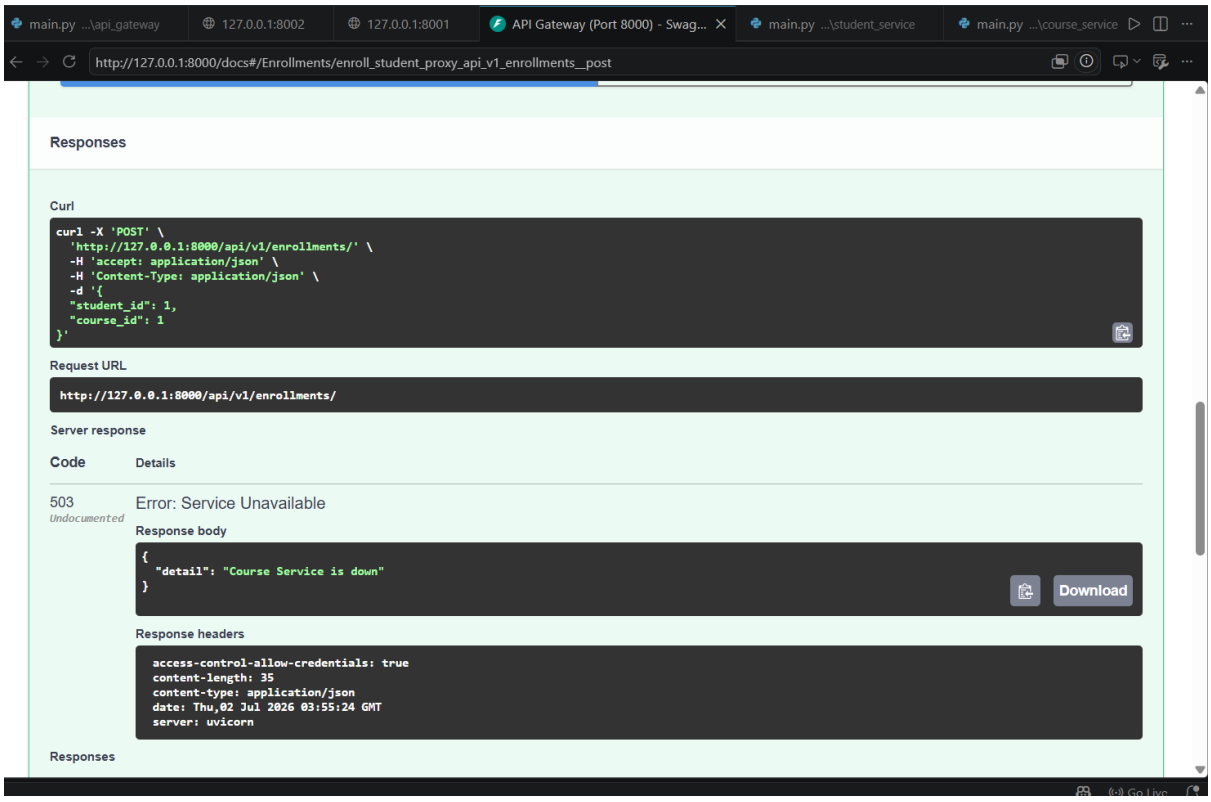
HANDS-ON 10 [Advanced]
Microservices Architecture — Concepts & Decomposition
Topics Covered:
Monolith vs Microservices✓
Service Decomposition✓
Inter-Service Communication✓
API Gateway Pattern✓
Service Discovery (concept)✓
Practising with Flask Microservices✓
In this final hands-on, you will understand why and when to break a monolith into microservices, decompose the Course Management system into services, implement basic inter-service communication, and understand the API Gateway pattern.
Task 1: Decompose the Monolith into Services
Goal: Analyse the Course Management API and identify natural microservice boundaries.
Steps:
96.
Review the Course Management API you built. Identify 3–4 bounded contexts — groups of related functionality that could be independent services. Document them in a README.md as: Service Name | Responsibility | Endpoints it owns | Database it owns.
97.
The natural decomposition: Student Service (student CRUD, enrollment), Course Service (department and course CRUD), Auth Service (registration, login, token validation), Notification Service (email confirmations).
98.
Create two minimal Flask apps in separate folders: course_service/ (port 5001) and student_service/ (port 5002). Each has its own database and its own subset of the original endpoints.
99.
Verify each service runs independently: python app.py in each folder. Confirm they do not share a database.
Hint:
•
The key microservices principle: each service owns its data. No service should directly query another service's database.
•
Start with 2 services for this exercise — do not over-engineer. Microservices add operational complexity; justify each split with a clear business or scaling reason.
Expected Outcome: Two Flask apps run on separate ports. Each has its own SQLite database. Their endpoints do not overlap.
Task 2: Inter-Service Communication and API Gateway Pattern

Goal: Implement synchronous inter-service calls and understand the API Gateway pattern.
Steps:
100.
Add an enrollment endpoint to Student Service: POST /api/students/{id}/enroll. This endpoint needs to verify the course exists — it must call Course Service's GET /api/courses/{id}/ using Python's requests library (pip install requests).
101.
Handle the scenario where Course Service is unavailable: catch the ConnectionError and return 503 Service Unavailable with a descriptive message.
102.
Create a simple API Gateway using Flask (gateway/ folder, port 5000): a single Flask app that proxies requests to the correct service. /api/courses/* → Course Service, /api/students/* → Student Service. Use requests.request() to forward calls.
103.
Test the full flow through the gateway: POST http://localhost:5000/api/students/1/enroll with a course_id — the gateway routes to Student Service, which calls Course Service to verify the course.
104.
Document in a README: what are the trade-offs of synchronous (HTTP) vs asynchronous (message queue) inter-service communication? When would you use a message queue like RabbitMQ or Kafka instead?

Hint:
•
Synchronous inter-service calls create tight coupling — if Course Service is down, enrollment fails. Message queues decouple services at the cost of eventual consistency.
•
A real API Gateway also handles auth, rate limiting, and SSL termination — your simple proxy demonstrates the routing concept without those production concerns.
Expected Outcome: POST to the gateway successfully routes through Student Service → Course Service. Stopping Course Service causes the enrollment endpoint to return 503.

OUTCOME:


main.py ...\api_gateway
127.0.0.1:8002
127.0.0.1:8001
API Gateway (Port 8000) - Swag...
main.py ...\student_service
main.py ...\course_service
http://127.0.0.1:8000/docs#/Enrollments/enroll_student_proxy_api_v1_enrollments__post
POST
/api/v1/enrollments/
Enroll Student Proxy
FAULT TOLERANT ENDPOINT: Returns 503 if the downstream Course Service is killed
Parameters
Cancel
Reset
No parameters
Request body
required
application/json
Edit Value
Schema
{
"student_id": 1,
"course_id": 1
}
Go Live


main.py ...\api_gateway
127.0.0.1:8002
127.0.0.1:8001
API Gateway (Port 8000) - Swag...
main.py ...\student_service
main.py ...\course_service
http://127.0.0.1:8000/docs#/Enrollments/enroll_student_proxy_api_v1_enrollments__post
Responses
Curl
curl -X 'POST' \
'http://127.0.0.1:8000/api/v1/enrollments/' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
"student_id": 1,
"course_id": 1
}'
Request URL
http://127.0.0.1:8000/api/v1/enrollments/
Server response
Code
Details
503
Undocumented
Error: Service Unavailable
Response body
{
"detail": "Course Service is down"
}
Download
Response headers
access-control-allow-credentials: true
content-length: 35
content-type: application/json
date: Thu,02 Jul 2026 03:55:24 GMT
server: uvicorn
Responses